



Top 'K' Frequent Numbers (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given an unsorted array of numbers, find the top 'K' frequently occurring numbers in it.

Example 1:

Input: [1, 3, 5, 12, 11, 12, 11], K = 2
Output: [12, 11]
Explanation: Both '11' and '12' appeared twice.

Example 2:

Input: [5, 12, 11, 3, 11], K = 2
Output: [11, 5] or [11, 12] or [11, 3]
Explanation: Only '11' appeared twice, all other numbers appeared once.

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class TopKFrequentNumbers {
4
5     public static List<Integer> findTopKFrequentNumbers(int[] nums, int k) {
6         List<Integer> topNumbers = new ArrayList<>(k);
7         // TODO: Write your code here
8         return topNumbers;
9     }
10
11     public static void main(String[] args) {
12         List<Integer> result = TopKFrequentNumbers.findTopKFrequentNumbers(new int[] { 1, 3, 5, 12, 11, 12, 11 }, 2);
13         System.out.println("Here are the K frequent numbers: " + result);
14     }
15 }
```

```

14
15     result = TopKFrequentNumbers.findTopKFrequentNumbers(new int[] { 5, 12, 11, 3, 11 }, 2);
16     System.out.println("Here are the K frequent numbers: " + result);
17 }
18 }
19

```

Run

Save

Reset



Solution

This problem follows [Top 'K' Numbers](#). The only difference is that in this problem, we need to find the most frequently occurring number compared to finding the largest numbers.

We can follow the same approach as discussed in the **Top K Elements** problem. However, in this problem, we first need to know the frequency of each number, for which we can use a **HashMap**. Once we have the frequency map, we can use a **Min Heap** to find the 'K' most frequently occurring number. In the **Min Heap**, instead of comparing numbers we will compare their frequencies in order to get frequently occurring numbers

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class TopKFrequentNumbers {
4
5      public static List<Integer> findTopKFrequentNumbers(int[] nums, int k) {
6          // find the frequency of each number
7          Map<Integer, Integer> numFrequencyMap = new HashMap<>();
8          for (int n : nums)
9              numFrequencyMap.put(n, numFrequencyMap.getOrDefault(n, 0) + 1);
10
11          PriorityQueue<Map.Entry<Integer, Integer>> minHeap = new PriorityQueue<Map.Entry<Integer, Integer>>(
12              (e1, e2) -> e1.getValue() - e2.getValue());
13
14          // go through all numbers of the numFrequencyMap and push them in the minHeap, which will have
15          // top k frequent numbers. If the heap size is more than k, we remove the smallest (top) number
16          for (Map.Entry<Integer, Integer> entry : numFrequencyMap.entrySet()) {
17              minHeap.add(entry);
18              if (minHeap.size() > k) {
19                  minHeap.poll();
20              }
21          }
22
23          // create a list of top k numbers
24          List<Integer> topNumbers = new ArrayList<>(k);
25          while (!minHeap.isEmpty()) {
26              topNumbers.add(minHeap.poll().getKey());
27          }
28          return topNumbers;
29      }
30
31      public static void main(String[] args) {
32          List<Integer> result = TopKFrequentNumbers.findTopKFrequentNumbers(new int[] { 1, 3, 5, 12, 11, 12, 1, 2 });
33          System.out.println("Here are the K frequent numbers: " + result);
34
35          result = TopKFrequentNumbers.findTopKFrequentNumbers(new int[] { 5, 12, 11, 3, 11 }, 2);
36          System.out.println("Here are the K frequent numbers: " + result);
37      }

```

```
38 }  
39
```

Run

Save

Reset

> Time complexity

The time complexity of the above algorithm is $O(N + N * \log K)$.

Space complexity

The space complexity will be $O(N)$. Even though we are storing only 'K' numbers in the heap. For the frequency map, however, we need to store all the 'N' numbers.

[< Back](#)[✓ Mark As Completed](#)[Next >](#)

Connect Ropes (easy)

Frequency Sort (medium)